

Maintainer Manual — Snoopy HQ Journal

Landed here first? This manual assumes the project is already running on your machine and that you know what an import package is. If either is not true, read [GETTING-STARTED.md](#) first — it takes about 45 minutes and covers setup, vocabulary and a narrated first article. All documents are routed from the [documentation index](#).

Everyday tasks, in order of how often you will do them. Copy-paste friendly. For *why* anything works this way, see [PROJECT-HANDBOOK.md](#).

Contents

0. Before you start
 1. The mental model in one picture
 2. Add a blog post (the easy way — Markdown)
 3. Add a blog post (full control — JSON)
 4. Where images and documents go, and how they attach
 5. Add or remove an asset on an article that is already published
 6. Edit a published article
 7. Remove a blog post
 8. Re-importing and `importId`
 9. Troubleshooting
 10. Before you commit
 11. Quick reference
-

o. Before you start

```
npm install    # first time only, or after pulling new dependencies
npm run dev    # starts the site AND the import watcher
```

Open <http://localhost:3000>.

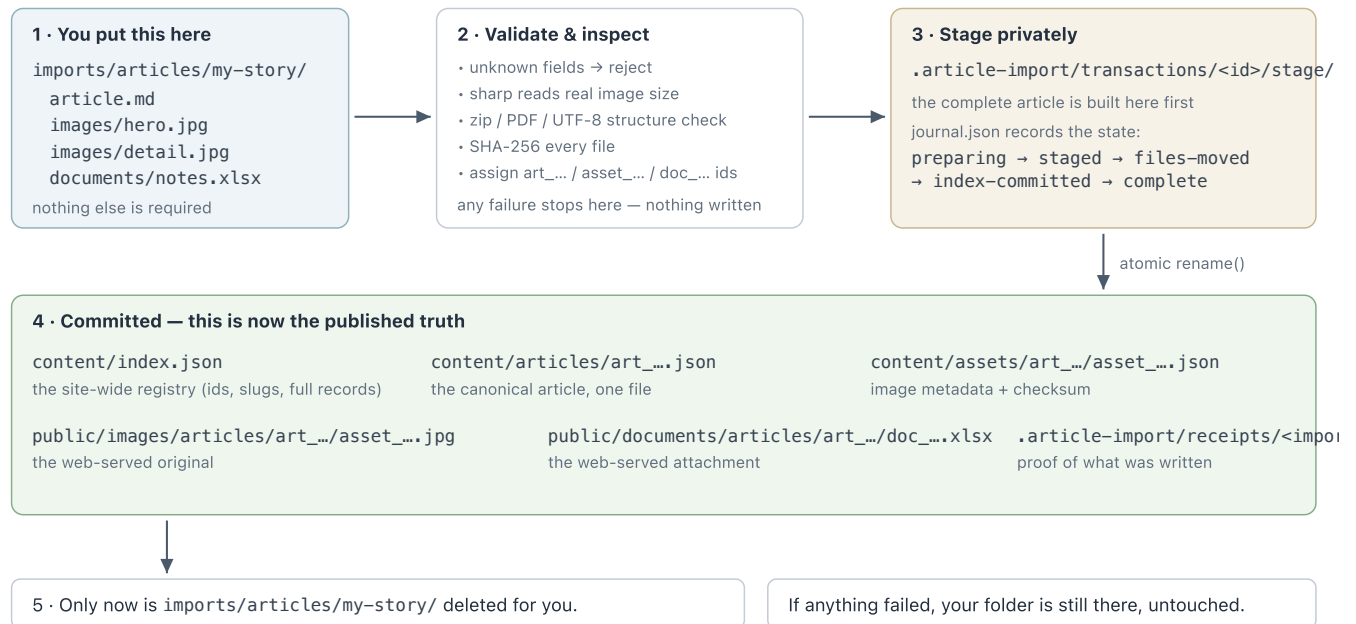
Leave `npm run dev` running while you work. It watches `imports/` and publishes anything you drop in there automatically.

Two golden rules

1. To publish, change, or move content — you edit **content files**, never components.
 2. Never hand-edit `content/index.json` while `npm run dev` is running.
-

1. The mental model in one picture

What happens when you drop a package into the inbox



You put a folder in `imports/articles/`. The importer validates it, copies everything to its permanent home, updates the registry, and **then** deletes your folder. If it deletes your folder, it worked. If your folder is still there, read the terminal — it says why.

2. Add a blog post (the easy way — Markdown)

Step 1 — make the folder

```
mkdir -p imports/articles/my-first-story/images
mkdir -p imports/articles/my-first-story/documents # only if you have attachments
```

Step 2 — put your files in

```
imports/articles/my-first-story/
├─ article.md
├─ images/
│   ├─ hero.jpg
│   └─ the-market.jpg
└─ documents/
    └─ timeline.xlsx
```

Step 3 — write `article.md`

```
---
title: A morning at the fish market
slug: a-morning-at-the-fish-market
category: Travel
date: 2026-08-05
author: Al Zadid Yusuf
description: Cold air, loud auctions and the best breakfast I have had in a year.
tags: [Travel, Japan, Food]
accent: sky
---
```

Getting there

The train left before sunrise and arrived while the city was still deciding whether to wake up.

![[Crates of ice at dawn](images/hero.jpg)]

The first crates being wheeled out, a little after five.

The auction floor is not open to everyone, and the queue starts earlier than you would like.

What to eat

1. **Tamago** Sweet, warm, and sold from a stall with no sign.
2. **Uni** Order it and stop asking questions.

[[Full morning timeline](documents/timeline.xlsx)]

Every stop, with the times that actually worked.

> The best meals are the ones you had to get up early for.

Sources

- [Market opening times](https://example.com/times)
- [How the auction works](https://example.com/auction)

Step 4 — publish

If `npm run dev` is running: **wait ~3 seconds**. The watcher imports it and the page reloads. Watch the terminal:

```
[article-watch] articles/my-first-story: imported (art_9f2c...)
```

Otherwise, run it yourself:

```
npm run import:articles:dry-run    # check first – changes nothing
npm run import:articles            # actually publish
```

Your post is now at <http://localhost:3000/blog/a-morning-at-the-fish-market> .

Markdown cheat sheet

You write	You get
<code>## Heading</code>	A new section (also becomes a table-of-contents entry)
A blank-line-separated block of text	A paragraph
<code>![alt text](images/foo.jpg)</code>	An image, placed right where you wrote it
A line of <code>*italic text*</code> directly after an image or document	That item's caption
<code>[Label](documents/foo.xlsx)</code> on its own line	An attachment card with Preview + Download
<code>1. / 2. lines</code>	A numbered list
<code>– lines</code>	A bulleted list
<code>– [Label](https://...)</code> – a bullet list where every item is a link	A "sources" link list instead of a bullet list
<code>> text</code>	A pull-quote at the end of the section
<code>**Bold** rest of line</code> inside a list item	A labelled list item

Frontmatter fields

Field	Required	Notes
<code>title</code>	✓	
<code>slug</code>	✓	lowercase-with-hyphens. This is the URL.
<code>category</code>	✓	Free text. A new category appears in "Pick a path" automatically.
<code>date</code>	✓	<code>2026-08-05</code> or <code>5 August 2026</code>
<code>author</code>	✓	
<code>description</code>	✓	(or <code>summary</code> / <code>dek</code>) Used on cards, search and social previews.
<code>tags</code>	—	<code>[0ne, Two]</code> . Defaults to the category.
<code>accent</code>	—	<code>sky</code> · <code>coral</code> · <code>teal</code> · <code>navy</code> . Card colour. Default <code>sky</code> .
<code>art</code>	—	<code>house</code> · <code>gift</code> · <code>shelf</code> · <code>type</code> · <code>weekend</code> · <code>city</code> . The illustration used when a post has no image.
<code>artLabel</code>	—	Text drawn inside that illustration.
<code>importId</code>	—	Defaults to <code>markdown-<slug>-v1</code> . See §8.

3. Add a blog post (full control — JSON)

Use this when you need exact image placement, per-section attachments, or links to related articles.

```
mkdir -p imports/articles/my-second-story/images
cp imports/articles/_template/article.json.example \
  imports/articles/my-second-story/article.json
```

(The `_template` folder itself is never imported — folders starting with `_` or `.` are skipped.)


```
{
  "schemaVersion": 1,
  "importId": "my-second-story-v1",
  "article": {
    "slug": "a-day-worth-remembering",
    "category": "Travel",
    "title": "A day worth remembering",
    "summary": "A concise summary used on cards, search and social previews.",
    "date": "5 August 2026",
    "author": "Al Zadid Yusuf",
    "tags": ["Travel", "Personal stories"],
    "accent": "sky",
    "art": "city",
    "heroAssetKey": "hero",
    "sections": [
      {
        "id": "the-first-stop",
        "title": "The first stop",
        "paragraphs": [
          "The first paragraph of the section.",
          "The second paragraph, followed by an image."
        ],
        "images": [{ "assetKey": "detail", "afterParagraph": 1 }],
        "documents": [{ "documentKey": "timeline", "afterParagraph": 1 }]
      }
    ],
    "relatedArticleIds": ["tokyo-skytree-and-shibuya"]
  },
  "assets": [
    {
      "key": "hero",
      "file": "images/hero.jpg",
      "title": "A wide opening view",
      "alt": "Describe what is visible, for readers who cannot see the image",
      "caption": "The caption shown beneath the image."
    },
    {
      "key": "detail",
      "file": "images/detail.jpg",
      "title": "A closer detail",
      "alt": "A clear description of the detail photograph",
      "caption": "A second caption tied to this exact asset.",
      "portrait": true
    }
  ]
}
```

```
],  
  "documents": [  
    {  
      "key": "timeline",  
      "file": "documents/timeline.xlsx",  
      "title": "The day, hour by hour",  
      "caption": "Every stop and the times that actually worked."  
    }  
  ]  
}
```

Then publish exactly as in §2 step 4.

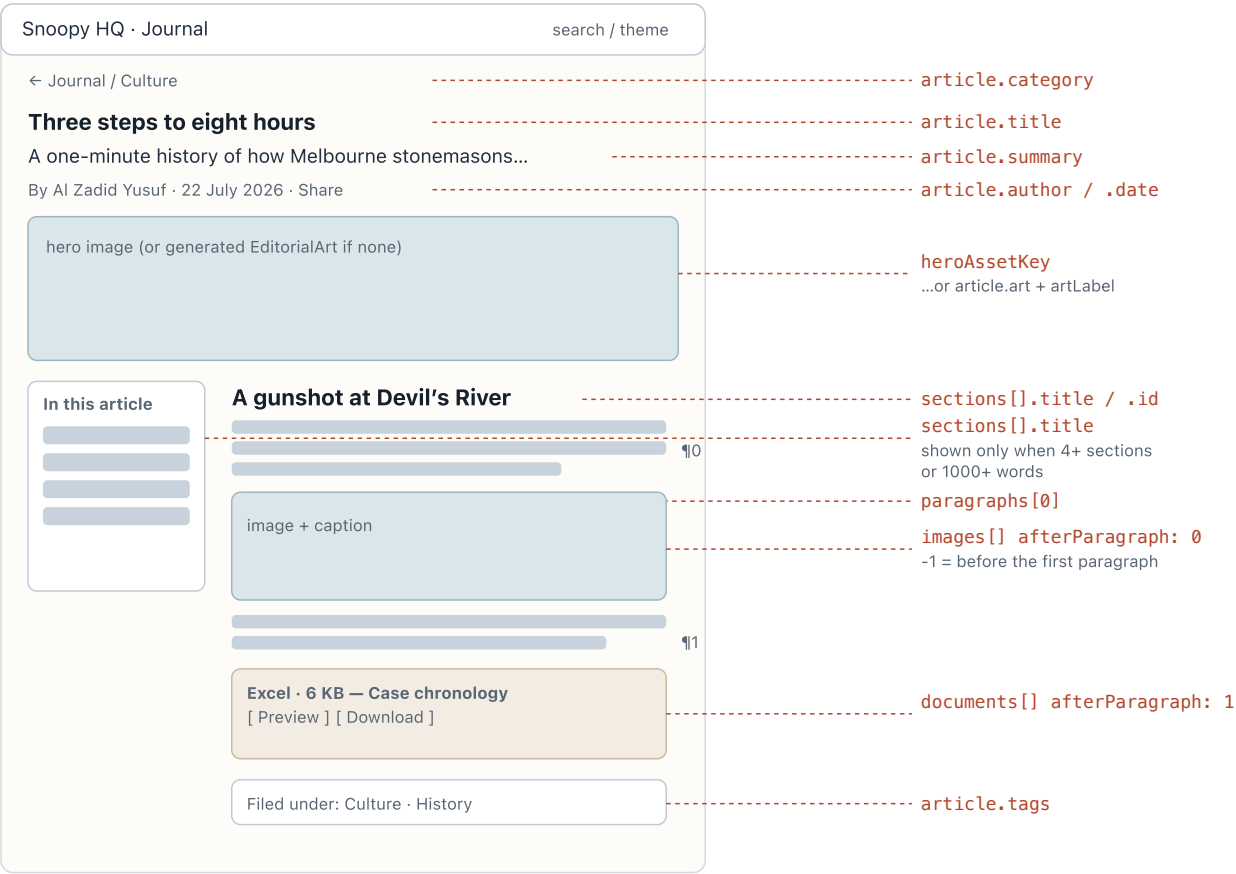
Two rules the validator will enforce:

- Every `key` in `assets` / `documents` must be placed somewhere in the article.
- Every `assetKey` / `documentKey` placed in the article must exist in `assets` / `documents` .

No orphans, no dangling references.

4. Where images and documents go, and how they attach

Anatomy of an article page — and the content field that controls each part



afterParagraph — the only positioning rule you need

Value	Where it renders
-1	Right after the section heading, before any text
0	After the first paragraph
1	After the second paragraph
n	After paragraph n+1

```
"paragraphs": ["First.", "Second.", "Third."],
"images": [
  { "assetKey": "top",    "afterParagraph": -1 },
  { "assetKey": "middle", "afterParagraph": 1  }
]
```

renders: image(top) → "First." → "Second." → image(middle) → "Third."

To move an image or attachment, change that one number and re-import (or, for an already-published article, edit the number in the two JSON files — see §6).

Image rules

Rule	Value
Formats	JPEG, PNG, WebP, AVIF
Max per file	25 MB
Max per package	250 MB total, 30 images
Max pixels	200 megapixels
<code>alt</code>	Required. Never publish without it.
Path	Must be relative and inside the package. No <code>../</code> , no absolute paths, no symlinks.

Add `"portrait": true` to a tall photo — it switches to the narrower layout.

You do **not** resize images yourself. `npm run dev` / `npm run build` generate AVIF and WebP at 480 / 768 / 1200 / 1600 px into `public/_optimized/` automatically.

Document (attachment) rules

Rule	Value
Formats	<code>.pptx</code> <code>.docx</code> <code>.xlsx</code> <code>.csv</code> <code>.pdf</code> — modern formats only
Rejected	<code>.ppt</code> <code>.doc</code> <code>.xls</code> , macro-enabled variants (<code>.docm</code> , <code>.xlsm</code> , ...)
Max per file	50 MB (CSV: 10 MB)
Max per package	12 documents
CSV	Must be valid UTF-8

Each attachment renders as a card with **Preview** (opens in the browser, no download) and **Download**.

5. Add or remove an asset on an article that is *already published*

The importer never overwrites. For a published article you edit the canonical files by hand. **Stop** `npm run dev` **first**.

You must keep **two** files in sync — they contain the same record:

- `content/articles/<articleId>.json` — the article on its own
- `content/index.json` → the matching entry inside its `"articles"` array

Adding an image

1. Put the file at `public/images/articles/<articleId>/<newAssetId>.jpg` (any stable name works; the `asset_...` convention just matches the importer).
2. Add the asset record to `content/index.json` → `"assets"`, and its id to `"assetIds"`:

```
{
  "id": "asset_market_dawn",
  "articleId": "art_9f2c...",
  "articleSlug": "a-morning-at-the-fish-market",
  "title": "Crates at dawn",
  "src": "/images/articles/art_9f2c.../asset_market_dawn.jpg",
  "width": 2000,
  "height": 1333,
  "alt": "Stacked crates of ice on a wet market floor before sunrise",
  "caption": "The first crates being wheeled out, a little after five."
}
```

`width` and `height` must be the **real** pixel dimensions. Check with:

```
npx --yes image-size public/images/articles/art_9f2c.../asset_market_dawn.jpg
```

3. Place it in the article by adding the same object **plus** `afterParagraph` to the section's `images` array — in *both* `content/articles/<articleId>.json` and the copy inside `content/index.json`.
4. `npm run optimize:images && npm run dev`

Removing an image

1. Delete it from the section's `images` array in both files.
2. Delete its entry from `"assets"` and its id from `"assetIds"` in `content/index.json`.
3. Delete the files:

```
rm public/images/articles/<articleId>/<assetId>.jpg
rm content/assets/<articleId>/<assetId>.json
rm -rf public/_optimized/images/articles/<articleId> # variants regenerate
```

4. `npm run dev` and check the article and the homepage card still look right.

Careful: the homepage card uses an article's **first** image. Deleting it changes which picture appears on the card.

Adding or removing an attachment

Same shape, different folders:

- file → `public/documents/articles/<articleId>/<docId>.xlsx`
- record → `content/index.json` → `"documents"` + `"documentIds"`
- placement → the section's `documents` array in both files

The record's `mimeType` **must** match the extension exactly, or the build fails:

Extension	mimeType
<code>.pdf</code>	<code>application/pdf</code>
<code>.csv</code>	<code>text/csv; charset=utf-8</code>
<code>.docx</code>	<code>application/vnd.openxmlformats-officedocument.wordprocessingml.document</code>
<code>.xlsx</code>	<code>application/vnd.openxmlformats-officedocument.spreadsheetml.sheet</code>
<code>.pptx</code>	<code>application/vnd.openxmlformats-officedocument.presentationml.presentation</code>

`size` must be the real byte count:

```
stat -f%z public/documents/articles/<articleId>/<docId>.xlsx
```

6. Edit a published article

Stop `npm run dev` first.

What you are editing	File
An imported article (id starts <code>art_...</code>)	<code>content/articles/<articleId>.json</code> and the same record in <code>content/index.json</code>
A built-in article (e.g. <code>tokyo-skytree-and-shibuya</code>)	<code>app/content/static-articles.ts</code>

Safe to change freely: `title`, `summary`, `paragraphs`, `quote`, `list`, `references`, `tags`, `category`, `accent`, `art`, `caption`, `alt`, `afterParagraph`.

Change with care:

Field	Why
<code>slug</code>	Breaks existing links. Also update any <code>related</code> entry that used the old slug.
<code>id</code>	Don't. Images, documents and related links all point at it.
<code>date</code>	Changes the article's position in the feed and what is "featured".
<code>sections[].id</code>	It is the <code>#anchor</code> — old links to that section break.

Then:

```
npm run dev    # confirm it renders
npm test       # confirm nothing else broke
```

7. Remove a blog post

There is no delete script yet. Do it in this order, with `npm run dev` **stopped**.

Say the article is `art_036d0a4179dd79db7d9dff25`, slug `devils-river-murder-1863`.

1. Delete its files

```
rm content/articles/art_036d0a4179dd79db7d9dff25.json
rm -rf content/assets/art_036d0a4179dd79db7d9dff25
rm -rf content/documents/art_036d0a4179dd79db7d9dff25
rm -rf public/images/articles/art_036d0a4179dd79db7d9dff25
rm -rf public/documents/articles/art_036d0a4179dd79db7d9dff25
rm -rf public/_optimized/images/articles/art_036d0a4179dd79db7d9dff25
```

2. Edit `content/index.json` — remove the article from **all** of these:

Key	Remove
<code>articleIds</code>	the <code>art_...</code> string
<code>slugs</code>	the slug string
<code>assetIds</code>	every <code>asset_...</code> belonging to it
<code>documentIds</code>	every <code>doc_...</code> belonging to it
<code>articles</code>	the object whose <code>id</code> matches
<code>assets</code>	every object whose <code>articleId</code> matches
<code>documents</code>	every object whose <code>articleId</code> matches
<code>imports</code>	the entry whose <code>articleId</code> matches

3. Remove inbound references — search for the id and slug across the repo:

```
grep -rn "art_036d0a4179dd79db7d9dff25|devils-river-murder-1863" content app
```

Delete any hits inside another article's `related` / `relatedArticleIds`, and any entry in `app/content/document-placements.ts`.

4. Verify

```
npm run build && npm test
```

A duplicate or dangling id fails the **build** with a message naming the id — that is the safety net working.

Removing a built-in article instead: delete its object from `app/content/static-articles.ts` , its images from `app/content/media-assets.ts` , its documents from `app/content/document-assets.ts` and `document-placements.ts` , then the files under `public/images/...` .

8. Re-importing and `importId`

Every import is remembered by its `importId`.

Situation	Result
Same <code>importId</code> , identical files	Importer confirms the article is intact and just removes the duplicate inbox folder. Safe.
Same <code>importId</code> , different files	✗ Import ID "..." was already used by a different package.
New <code>importId</code> , but same <code>slug</code>	✗ Article slug "..." already exists.

To genuinely re-publish a changed article: **delete the old one** (§7), then import with a bumped id (`my-story-v2`).

9. Troubleshooting

Symptom	Cause and fix
Folder still sitting in <code>imports/</code> , nothing happened	Read the terminal. The importer prints the exact field and reason. Fix it — the watcher retries only after a file changes.
<code>contains the unsupported field "sumary"</code>	Typo in a field name. Unknown fields are always rejected.
Asset key <code>"x"</code> is declared but never placed	You listed an image but never referenced it in a section.
Article references undeclared asset key <code>"x"</code>	The reverse — placed but not declared.
<code>article.date</code> must use a real date	Use <code>5 August 2026</code> or <code>2026-08-05</code> .
Another article import is already running	A crashed run left a lock. <code>rm .article-import/import.lock</code>
Duplicate article id / Duplicate article slug on build	Two articles collide. Check <code>content/index.json</code> against <code>app/content/static-articles.ts</code> .
Images 404 or the site looks unstyled after a fresh clone	Generated folders are missing. Run <code>npm run dev</code> (or <code>npm run build</code>) once.
Document Preview spins forever or shows the retry panel	Usually a corrupt or legacy-format file. Confirm it opens in Office; check the browser console.
Post published but not on the homepage	Ordering is by <code>date</code> , newest first. An old date puts it in "From the archive", not "Start with these stories".
Changed <code>content/index.json</code> and nothing updated	Restart <code>npm run dev</code> .

Watcher status, if you want to see what it is doing:

```
cat .article-import/watcher.json
```

10. Before you commit

```
npm run lint
npm test          # builds the site, then runs both test suites
```

Commit these:

```
content/index.json
content/articles/**
content/assets/**
public/images/**
public/documents/**
app/**          (only if you changed code or built-in content)
```

Never commit these (they are gitignored and regenerate themselves):

```
public/_optimized/   public/vendor/   dist/   .article-import/   .local-test-assets/
```

11. Quick reference

```
npm run dev           # site + auto-import watcher
npm run dev:site      # site only
npm run import:articles:dry-run # validate the inbox, change nothing
npm run import:articles # import + regenerate image variants
npm run optimize:images # regenerate image variants only
npm run build         # production build
npm test              # build + all tests
npm run lint
```

I want to...	Go to
Publish a post	<code>imports/articles/<name>/article.md</code>
Move an image inside a post	its <code>afterParagraph</code>
Change a card's colour	the post's <code>accent</code>
Change a post's URL	its <code>slug</code> (and any <code>related</code> pointing at it)
Reorder the category menu	<code>app/content/categories.ts</code> → <code>preferredCategories</code>
Change site colours	<code>app/globals.css</code> → <code>:root</code> and <code>:root[data-theme="dark"]</code>
Change which posts are "featured"	<code>app/content/articles.ts</code> → <code>posts.slice(0, 3)</code>
Change when the table of contents appears	<code>app/components/article-view.tsx</code> → <code>hasTableOfContents</code>

A note on the pictures in this manual

The two diagrams above are annotated schematics, drawn to map each on-screen element to the exact content field that controls it — which a raw screenshot cannot do. To capture real screenshots for a specific change, run `npm run dev` and use your browser's own capture (macOS: `⌘⇧4`, then `Space` to grab a window), and drop the PNGs into `docs/images/`.